

DS Written Homework 3

Student ID: b10201054

May 1, 2025

1 Splay Tree

2 Time complexity of AVL tree and Splay tree

Search	Binary Search Tree	Splay Tree	AVL Tree
Worst Case	$O(n)$	Amortized $O(\log n)$ / $O(n)$	$O(\log n)$
Average	$O(n)$	$O(\log n)$	$O(\log n)$

Table 1: Time complexity of search in different trees

3 Data Structure for Different Scenarios

- (a) Splay tree; The root of splay tree is last accessed node. Since it frequently accesses a few popular keywords, the height of those keywords will be near the root, which helps us find them quickly.
- (b) Queue; Queue has the property 'first in first out'. Since we need to print jobs in the order they are received, 'first in first out' is what we need the most.
- (c) AVL tree; AVL tree is a balanced BST, whose search, insert, delete have the steady time complexity $O(\log n)$ even in the worst case.
- (d) Stack; Undo means to cancel the last step we have done. Stack has the property 'last-in, first-out', which matches the undo function needs.

4 AVL Tree

5 Amortized Analysis

First we named the stack S , with its size $|S| = n$.

- (a) The worst case time complexity of MultiPop is $\min(n, k)$. We know that the time complexity of Pop is $O(1)$. MultiPop(k) repeats $\min(n, k)$ times Pop, hence the time complexity of MultiPop(k) is $\min(n, k) \times O(1) = O(\min(n, k))$

- (b) Suppose that there are $n_{push}, n_{pop}, n_{multipop}$ times of Push, Pop, MultiPop to execute. Note that the legal Pop or MultiPop can only execute when the stack is not empty, we can find that the Push operation needs to be executed same as or more than Pop + MultiPop. That is, $n_{pop} + n_{multipop} \leq n_{push}$. The total time complexity is $T(n) = n_{push} + n_{pop} + n_{multipop} \leq 2 \cdot n_{push} = O(n)$. Hence, the amortized time complexity of an operation is $O(n)/n = O(1)$.

6 Algorithm Design

```

Input: x, T.root
Output: the successor of x
1 n = T.root succ = None while n is not None do
2   if  $x < n.value$  then
3     succ = n.value
4     n = n.left
5   else if  $x \geq n.value$  then
6     n = n.right
7 end
8 if succ is not None then
9   return succ
10 else
11   return "cannot find successor in the tree"

```

Algorithm 1: Find successor